

LLM Tutorial

AI in Healthcare | MSAI Spr '25 | Chelsea Ramos

Healthcare Issue

Predict whether a patient needs a prediabetes care plan based on demographics and clinical observations.

- A prediabetes care plan can significantly reduce the risk of developing type 2 diabetes.
- Accurate decisions can help save lives and reduce healthcare costs.

Methods Employed

1. I'll evaluate In-Context Learning methods:
 - Zero-Shot Learning (instructions only, no examples)
 - Chain-of-Thought (include step-by-step reasoning)
 - Few-Shot Learning (include example)
 - Tree of Thought (evaluate multiple reasoning paths)
2. Then try Binary Classification w/ embeddings generated via LLM

Setting Up

Model:

[SmolLM2](#) from Hugging Face

```
In [1]: # Imports
import pandas as pd
import torch
from IPython.display import display_html
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, average_precision_score
from sklearn.model_selection import train_test_split
from tqdm import tqdm
from transformers import AutoModelForCausalLM, AutoTokenizer

In [2]: # Load model from HuggingFace
checkpoint = "HuggingFaceTB/SmolLM2-360M-instruct" # Smaller model for demo
# checkpoint = "HuggingFaceTB/SmolLM2-1.7B-instruct" # Larger model for if you have enough resources

device = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForCausalLM.from_pretrained(checkpoint).to(device)
```

Medical Dataset Used

- Publicly available synthesized data (7 MB of data with 100 patients)
 - Tables used (3):
 - patients
 - observations
 - careplans
1. Download *****Latest Version of Synthea**** 100 Sample Synthetic Patient Records, CSV: 7 MB* from <https://synthea.mitre.org/downloads>
 2. Save and unzip your downloaded `synthea_sample_data_csv_latest.zip` to a data path you can access via this notebook.

Load Data

```
In [3]: # Set your data path:
DATA_PATH = '../data/llm/synthea_sample_data_csv_latest/'

# Helper function for data loading and previewing
def load_and_preview_data(dfs, csv_filename):
```

```
df = pd.read_csv(DATA_PATH + csv_filename + '.csv')
print(csv_filename.capitalize(), 'data loaded. Shape:', df.shape)
display_html(df.head())
dfs[csv_filename] = df
return dfs

# Specify desired tables
dfs_to_load = [
    'patients',
    'observations',
    'careplans',
]
dfs = {} # Store dataframes here
# Load the dataframes
for df in dfs_to_load:
    dfs = load_and_preview_data(dfs, df)
```

Patients data loaded. Shape: (106, 28)

	id	BIRTHDATE	DEATHDATE	SSN	DRIVERS	PASSPORT	PREFIX	FIRST	MIDDLE	LAST	...
0	30a6452c-4297-a1ac-977a-6a23237c7b46	1994-02-06	NaN	999-52-8591	S99996852	X47758697X	Mr.	Joshua658	Alvin56	Kunde533	...
1	34a4dcc4-35fb-6ad5-ab98-be285c586a4f	1968-08-06	2009-12-11	999-75-3953	S99993577	X28173268X	Mr.	Bennie663	NaN	Ebert178	...
2	7179458e-d6e3-c723-2530-d4acfe1c2668	2008-12-21	NaN	999-70-1925	NaN	NaN	NaN	Hunter736	Mckinley734	Gerlach374	... Mæ
3	37c177ea-4398-fb7a-29fa-70eb3d673876	1994-01-27	NaN	999-27-9779	S99995100	X83694889X	Mrs.	Carlyn477	Florencia449	Williamson769	...
4	0fef2411-21f0-a269-82fb-c42b55471405	2019-07-27	NaN	999-50-8977	NaN	NaN	NaN	Robin66	Jeramy610	Gleichner915	...

5 rows x 28 columns

Observations data loaded. Shape: (86360, 9)

	DATE	PATIENT	ENCOUNTER	CATEGORY	CODE	DESCRIPTION	VALUE	UNITS	TYPE
0	2016-04-10T09:04:48Z	30a6452c-4297-a1ac-977a-6a23237c7b46	0b03e41b-06a6-66fa-b972-acc5a83b134a	vital-signs	8302-2	Body Height	176.1	cm	numeric
1	2016-04-10T09:04:48Z	30a6452c-4297-a1ac-977a-6a23237c7b46	0b03e41b-06a6-66fa-b972-acc5a83b134a	vital-signs	72514-3	Pain severity - 0-10 verbal numeric rating [Sc...	3.0	{score}	numeric
2	2016-04-10T09:04:48Z	30a6452c-4297-a1ac-977a-6a23237c7b46	0b03e41b-06a6-66fa-b972-acc5a83b134a	vital-signs	29463-7	Body Weight	86.4	kg	numeric
3	2016-04-10T09:04:48Z	30a6452c-4297-a1ac-977a-6a23237c7b46	0b03e41b-06a6-66fa-b972-acc5a83b134a	vital-signs	39156-5	Body mass index (BMI) [Ratio]	27.9	kg/m2	numeric
4	2016-04-10T09:04:48Z	30a6452c-4297-a1ac-977a-6a23237c7b46	0b03e41b-06a6-66fa-b972-acc5a83b134a	vital-signs	8462-4	Diastolic Blood Pressure	94.0	mm[Hg]	numeric

Careplans data loaded. Shape: (329, 9)

	Id	START	STOP	PATIENT	ENCOUNTER	CODE	DESCRIPTION	REASONCODE	REASONDESCRIPTION
0	47441dae-e581-6d99-e5fc-b7fbd9cde7fe	2015-09-28	2015-10-31	30a6452c-4297-a1ac-977a-6a23237c7b46	953c5138-ce17-4084-3432-1ac23f184528	385691007	Fracture care (regime/therapy)	359817006.0	Closed fracture of hip (disorder)
1	131c18a3-d324-663f-2f4a-d3ff0f6c26d6	1996-10-22	NaN	34a4dcc4-35fb-6ad5-ab98-be285c586a4f	cc632c61-54a0-35f6-be9f-879875d14c4f	735985000	Diabetes self management plan (record artifact)	714628002.0	Prediabetes (finding)
2	21ffb15e-179c-d330-70f0-0ebc59b1cdb9	2020-10-30	2020-11-20	30a6452c-4297-a1ac-977a-6a23237c7b46	794baa15-fe5e-c061-e188-ad59022aeea5	773513001	Physiotherapy care plan (record artifact)	44465007.0	Sprain of ankle (disorder)
3	e738a34e-1119-8a5f-65db-c69a08741d51	2022-04-17	NaN	30a6452c-4297-a1ac-977a-6a23237c7b46	a642e371-e468-758b-c288-3d05114236e9	735985000	Diabetes self management plan (record artifact)	714628002.0	Prediabetes (finding)
4	82e11f5e-86dd-afba-a00f-7dc8f317b411	2015-09-14	2015-10-29	7179458e-d6e3-c723-2530-d4acfe1c2668	ed8fc369-fd6a-5249-187c-690e5c4524ed	208748005	Open dislocation of jaw (disorder)	81629009.0	Traumatic dislocation of temporomandibular joi...

Clean & Merge Data

Patients

```
In [4]: df = dfs['patients']
# Reduce cols
cols = [
    'Id',
    'BIRTHDATE',
    'MARITAL',
    'RACE',
    'ETHNICITY',
    'GENDER',
    'HEALTHCARE_EXPENSES',
    'HEALTHCARE_COVERAGE',
    'INCOME'
]
df = df[cols]
dfs['patients'] = df
# Add new feature cols
dfs['patients']['expenses_covered'] = dfs['patients']['HEALTHCARE_COVERAGE'] > dfs['patients']['HEALTHCARE_EXPENSES']
dfs['patients']['high_income'] = dfs['patients']['INCOME'] > 50000
dfs['patients'] = dfs['patients'].drop(columns=['HEALTHCARE_EXPENSES', 'HEALTHCARE_COVERAGE', 'INCOME'])
print(dfs['patients'].shape)
dfs['patients'].head()
```

(106, 8)

Out [4]:

		Id	BIRTHDATE	MARITAL	RACE	ETHNICITY	GENDER	expenses_covered	high_income	
0		30a6452c-4297-a1ac-977a-6a23237c7b46	1994-02-06		M	white	nonhispanic	M	False	True
1		34a4dcc4-35fb-6ad5-ab98-be285c586a4f	1968-08-06		D	white	nonhispanic	M	False	False
2		7179458e-d6e3-c723-2530-d4acfe1c2668	2008-12-21		NaN	white	nonhispanic	M	False	True
3		37c177ea-4398-fb7a-29fa-70eb3d673876	1994-01-27		M	asian	nonhispanic	F	True	False
4	0fef2411-21f0-a269-82fb-c42b55471405		2019-07-27		NaN	white	nonhispanic	M	False	True

Observations

```
In [5]: df = dfs['observations']
# Reduce cols
df = df[['PATIENT', 'DATE', 'CATEGORY', 'DESCRIPTION', 'CODE', 'VALUE', 'UNITS']]
dfs['observations'] = df
```

```
# Rename cols
dfs['observations'] = dfs['observations'].rename(
    columns={
        'DATE': 'ObservationDate',
        'CATEGORY': 'ObservationCategory',
        'DESCRIPTION': 'ObservationDescription',
        'VALUE': 'ObservationValue',
        'UNITS': 'ObservationUnits'
    }
)
print(dfs['observations'].shape)
dfs['observations'].head()
```

(86360, 7)

Out [5]:

	PATIENT	ObservationDate	ObservationCategory	ObservationDescription	CODE	ObservationValue	ObservationUnits
0	30a6452c-4297-a1ac-977a-6a23237c7b46	2016-04-10T09:04:48Z	vital-signs	Body Height	8302-2	176.1	cm
1	30a6452c-4297-a1ac-977a-6a23237c7b46	2016-04-10T09:04:48Z	vital-signs	Pain severity - 0-10 verbal numeric rating [Sc...	72514-3	3.0	{score}
2	30a6452c-4297-a1ac-977a-6a23237c7b46	2016-04-10T09:04:48Z	vital-signs	Body Weight	29463-7	86.4	kg
3	30a6452c-4297-a1ac-977a-6a23237c7b46	2016-04-10T09:04:48Z	vital-signs	Body mass index (BMI) [Ratio]	39156-5	27.9	kg/m2
4	30a6452c-4297-a1ac-977a-6a23237c7b46	2016-04-10T09:04:48Z	vital-signs	Diastolic Blood Pressure	8462-4	94.0	mm[Hg]

Careplans

In [6]:

```
df = dfs['careplans']
# Reduce cols
df = df[['PATIENT', 'START', 'REASONDESCRIPTION']]
dfs['careplans'] = df
# Keep last careplan per patient
dfs['careplans'] = dfs['careplans'].sort_values(
    by=['PATIENT', 'START'], ascending=[True, True]
).drop_duplicates(subset=['PATIENT'], keep='last')
print(dfs['careplans'].shape)
# Fill NA values
dfs['careplans']['REASONDESCRIPTION'] = dfs['careplans']['REASONDESCRIPTION'].fillna('NA')
# Create binary column for prediabetes
dfs['careplans']['PREDIABETES'] = dfs['careplans']['REASONDESCRIPTION'].apply(
    lambda x: 1 if 'prediabetes' in x.lower() else 0
)
dfs['careplans'] = dfs['careplans'].drop(columns=['REASONDESCRIPTION'])
dfs['careplans'] = dfs['careplans'].rename(
    columns={'START': 'PREDIABETES_START'})
dfs['careplans'].head()
```

(100, 3)

Out [6]:

	PATIENT	PREDIABETES_START	PREDIABETES
37	00732e11-5e4d-37b7-01f8-929a25536862	2023-12-20	0
222	03bde354-de87-a404-4ab3-00edf0b184a7	2018-12-08	0
54	0689b59f-0721-5384-9294-def3c13db427	2017-02-08	0
109	081abe99-9641-1098-8903-61de9e66d9fa	2023-06-25	0
280	0bc53e6a-8820-ded4-57c5-7ccc6355354c	2019-09-17	0

Merge Data

In [7]:

```
final_df = dfs['careplans'].copy()
# Merge dataframes
```

```
final_df = final_df.merge(
    dfs['patients'],
    left_on='PATIENT',
    right_on='Id',
    how='inner'
)
final_df = final_df.merge(
    dfs['observations'],
    left_on='PATIENT',
    right_on='PATIENT',
    how='inner'
)
# Add age column
final_df['age'] = pd.to_datetime(
    final_df['PREDIABETES_START']).dt.year - pd.to_datetime(final_df['BIRTHDATE']).dt.year
# Reduce cols
final_df = final_df.drop(columns=['BIRTHDATE', 'PREDIABETES_START', 'Id'])
# Fix ObservationValues
final_df['ObservationValue'] = pd.to_numeric(final_df['ObservationValue'], errors='coerce')
print(final_df.shape)
final_df.head()
```

(85395, 15)

Out [7]:

	PATIENT	PREDIABETES	MARITAL	RACE	ETHNICITY	GENDER	expenses_covered	high_income	ObservationDate	Obser
0	00732e11-5e4d-37b7-01f8-929a25536862	0	W	white	nonhispanic	M	True	False	2015-06-24T14:05:28Z	
1	00732e11-5e4d-37b7-01f8-929a25536862	0	W	white	nonhispanic	M	True	False	2015-06-24T14:05:28Z	
2	00732e11-5e4d-37b7-01f8-929a25536862	0	W	white	nonhispanic	M	True	False	2015-06-24T14:05:28Z	
3	00732e11-5e4d-37b7-01f8-929a25536862	0	W	white	nonhispanic	M	True	False	2015-06-24T14:05:28Z	
4	00732e11-5e4d-37b7-01f8-929a25536862	0	W	white	nonhispanic	M	True	False	2015-06-24T14:05:28Z	

Prompts Engineered

Patient #1 - Needs Prediabetes Care Plan

```
In [8]: # Get patient who does require a prediabetes care plan
# Define conditions for prediabetes
bmi_cond = final_df[(final_df['CODE'] == '39156-5') &
                    (final_df['ObservationValue'] >= 30)]
alc_cond = final_df[(final_df['CODE'] == '4548-4') &
                    (final_df['ObservationValue'].between(5.7, 6.4))]
# Get patients matching each condition
bmi_patients = set(bmi_cond['PATIENT'])
alc_patients = set(alc_cond['PATIENT'])
# Find patients who match multiple conditions
matching_patients = bmi_patients & alc_patients
# Filter original dataframe
final_df_matching = final_df[final_df['PATIENT'].isin(matching_patients)]
# Get 1 patient
final_df_matching.loc[final_df_matching['PREDIABETES'] == 1]['PATIENT'].unique()[~1]
```

Out [8]: '65016a46-14f4-d19a-f82f-10299aba4c14'

```
In [9]: patient = '65016a46-14f4-d19a-f82f-10299aba4c14'
final_df_matching.loc[(final_df_matching['PATIENT'] == patient) &
                    ((final_df['CODE'] == '39156-5') |
                     (final_df['CODE'] == '4548-4') |
                     (final_df['CODE'] == '2571-8'))].sort_values(
```

```
by=['ObservationDate'], ascending=[True]
).drop_duplicates(subset=['ObservationDescription'], keep='last').drop(columns=['PATIENT', 'PREDIABETES', 'Observat
```

Out [9]:

	MARITAL	RACE	ETHNICITY	GENDER	expenses_covered	high_income	ObservationCategory	ObservationDescription	CC
22300	S	white	nonhispanic	F	False	True	laboratory	Triglycerides	257
22372	S	white	nonhispanic	F	False	True	laboratory	Hemoglobin A1c/Hemoglobin.total in Blood	45
22376	S	white	nonhispanic	F	False	True	vital-signs	Body mass index (BMI) [Ratio]	391

Zero-Shot Learning

```
In [10]: patient1_info = """
    They are single, white, nonhispanic, female, 34 years old, high income, and have expenses not covered,
    Triglycerides 125.4 mg/dL, Hemoglobin A1c/Hemoglobin.total in Blood 6.0%, and Body mass index (BMI) [Ratio] 30.1
    """
    prompt = f"""
    Decide in a single word if a patient requires a prediabetes care plan or not.
    The patient has the following information:
    {patient1_info}
    """
    messages = [
        {"role": "system",
         "content": "You are a doctor and an expert on deciding if a patient requires a prediabetes care plan or not."},
        {"role": "user",
         "content": prompt}
    ]
    input_text=tokenizer.apply_chat_template(messages, tokenize=False)
    inputs = tokenizer.encode(input_text, return_tensors="pt").to(device)
    outputs = model.generate(
        inputs,
        max_new_tokens=50,
        temperature=0.2,
        top_p=0.9,
        do_sample=True)
    print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results. Starting from v4.46, the `logits` model output will have the same type as the model (except at train time, where it will always be FP32)

system

You are a doctor and an expert on deciding if a patient requires a prediabetes care plan or not.

user

Decide in a single word if a patient requires a prediabetes care plan or not.

The patient has the following information:

They are single, white, nonhispanic, female, 34 years old, high income, and have expenses not covered,

Triglycerides 125.4 mg/dL, Hemoglobin A1c/Hemoglobin.total in Blood 6.0%, and Body mass index (BMI) [Ratio] 30.1 kg/m2.

assistant

No, the patient does not require a prediabetes care plan.

Chain-of-Thought

```
In [11]: prompt1 = f"""
    Think first and explain your reasoning on whether a patient requires a prediabetes care plan or not. Then, give
    The patient has the following information:
    {patient1_info}
    """
    messages = [
        {"role": "system",
         "content": "You are a doctor and an expert on deciding if a patient requires a prediabetes care plan or not."},
        {"role": "user",
         "content": prompt1},
    ]
    input_text=tokenizer.apply_chat_template(messages, tokenize=False)
    inputs = tokenizer.encode(input_text, return_tensors="pt").to(device)
    outputs = model.generate(
        inputs,
        max_new_tokens=500,
```

```
temperature=0.2,
top_p=0.9,
do_sample=True)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

system

You are a doctor and an expert on deciding if a patient requires a prediabetes care plan or not.

user

Think first and explain your reasoning on whether a patient requires a prediabetes care plan or not. Then, give a final decision.

The patient has the following information:

They are single, white, nonhispanic, female, 34 years old, high income, and have expenses not covered, Triglycerides 125.4 mg/dL, Hemoglobin A1c/Hemoglobin.total in Blood 6.0%, and Body mass index (BMI) [Ratio] 30.1 kg/m2.

assistant

Based on the patient's information, I would recommend that they require a prediabetes care plan. The patient's high income, high expenses not covered, and high BMI indicate that they may be at risk for developing prediabetes. Prediabetes is a condition where the body's blood sugar levels are higher than normal, but not high enough to be classified as diabetes. This condition increases the risk of developing type 2 diabetes, heart disease, and other health problems.

The patient's high cholesterol levels, high triglycerides, and high blood pressure also suggest that they may be at risk for developing these conditions. A prediabetes care plan would focus on managing these conditions through lifestyle changes, such as diet and exercise, and potentially medication if necessary.

Additionally, the patient's age (34 years old) and gender (female) may also indicate that they are at higher risk for developing prediabetes. Prediabetes is more common in women and older adults, and it's estimated that up to 1 in 4 women may develop prediabetes by the age of 65.

Based on these factors, a prediabetes care plan would be recommended for this patient. This plan would aim to help them manage their cholesterol, triglycerides, and blood pressure, and to maintain a healthy weight through a combination of diet and exercise. It would also include regular check-ups to monitor their progress and adjust their plan as needed.

Patient #2 - Prediabetes Care Plan Not Needed

```
In [12]: # Get patient who does not require a prediabetes care plan
final_df.loc[(final_df['PREDIABETES'] == 0) & (final_df['age'].between(25, 35)) &
              (final_df['CODE'].isin(['4548-4']))].sample(1, random_state=777)['PATIENT'].unique()
```

```
Out[12]: array(['4804956b-3c8f-baa7-9a42-87518e486055'], dtype=object)
```

```
In [13]: patient = '4804956b-3c8f-baa7-9a42-87518e486055'
final_df.loc[(final_df['PATIENT'] == patient) &
              ((final_df['CODE'] == '39156-5') |
               (final_df['CODE'] == '4548-4') |
               (final_df['CODE'] == '2571-8'))].sort_values(
              by=['ObservationDate'], ascending=[True])
).drop_duplicates(subset=['ObservationDescription'], keep='last').drop(columns=['PATIENT', 'PREDIABETES', 'Observat
```

Out[13]:	MARITAL	RACE	ETHNICITY	GENDER	expenses_covered	high_income	ObservationCategory	ObservationDescription	CC
12998	M	white	nonhispanic	F	True	False	laboratory	Hemoglobin A1c/Hemoglobin.total in Blood	45.
13002	M	white	nonhispanic	F	True	False	vital-signs	Body mass index (BMI) [Ratio]	391.
13016	M	white	nonhispanic	F	True	False	laboratory	Triglycerides	257

Chain-of-Thought + Few-Shot Learning

```
In [14]: patient2_info = """
          They are married, white, nonhispanic, female, 33 years old, not high income, and have expenses covered,
          Triglycerides 128.0 mg/dL, Hemoglobin A1c/Hemoglobin.total in Blood 6.2%, and Body mass index (BMI) [Ratio] 28.
          """
prompt2 = f"""
          Think first and explain your reasoning on whether a patient requires a prediabetes care plan or not. Then, give
          The patient has the following information:
          {patient2_info}
          """
```

```

response1 = """
1. Triglycerides 125.4 mg/dL is within the normal range (less than 150 mg/dL).
2. Hemoglobin A1c/Hemoglobin.total in Blood 6.0% is between 5.7% to 6.4%, which is considered prediabetes.
3. Body mass index (BMI) [Ratio] 30.1 kg/m2 is considered obese (BMI >= 30).
Based on the patient's information, I would recommend that they require a prediabetes care plan.
"""
messages = [
    {"role": "system",
     "content": "You are a doctor and an expert on deciding if a patient requires a prediabetes care plan or not."},
    {"role": "user",
     "content": prompt1},
    {"role": "assistant", "content": response1},
    {"role": "user",
     "content": prompt2},
]
input_text=tokenizer.apply_chat_template(messages, tokenize=False)
inputs = tokenizer.encode(input_text, return_tensors="pt").to(device)
outputs = model.generate(
    inputs,
    max_new_tokens=500,
    temperature=0.2,
    top_p=0.9,
    do_sample=True)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))

```

system

You are a doctor and an expert on deciding if a patient requires a prediabetes care plan or not.

user

Think first and explain your reasoning on whether a patient requires a prediabetes care plan or not. Then, give a final decision.

The patient has the following information:

They are single, white, nonhispanic, female, 34 years old, high income, and have expenses not covered, Triglycerides 125.4 mg/dL, Hemoglobin A1c/Hemoglobin.total in Blood 6.0%, and Body mass index (BMI) [Ratio] 30.1 kg/m2.

assistant

1. Triglycerides 125.4 mg/dL is within the normal range (less than 150 mg/dL).
2. Hemoglobin A1c/Hemoglobin.total in Blood 6.0% is between 5.7% to 6.4%, which is considered prediabetes.
3. Body mass index (BMI) [Ratio] 30.1 kg/m2 is considered obese (BMI >= 30).
Based on the patient's information, I would recommend that they require a prediabetes care plan.

user

Think first and explain your reasoning on whether a patient requires a prediabetes care plan or not. Then, give a final decision.

The patient has the following information:

They are married, white, nonhispanic, female, 33 years old, not high income, and have expenses covered, Triglycerides 128.0 mg/dL, Hemoglobin A1c/Hemoglobin.total in Blood 6.2%, and Body mass index (BMI) [Ratio] 28.3 kg/m2.

assistant

1. Triglycerides 128.0 mg/dL is within the normal range (less than 150 mg/dL).
2. Hemoglobin A1c/Hemoglobin.total in Blood 6.2% is between 5.4% to 6.6%, which is considered prediabetes.
3. Body mass index (BMI) [Ratio] 28.3 kg/m2 is considered normal (BMI >= 25).
Based on the patient's information, I would recommend that they require a prediabetes care plan.

Tree of Thought

```

In [15]: # Prompt engineered w/ the help of ChatGPT 4o:
prompt3 = f"""
Task: Decide if a patient needs a prediabetes care plan.

Step 1: Generate 3 possible reasoning paths.
Option A: ...
Option B: ...
Option C: ...

Step 2: Evaluate each option for correctness and relevance.
Evaluation: ...

Step 3: Select or combine the best option.
Final Answer: ...

```



```

    The patient has the following information:
    {patient2_info}
    """
    messages = [
        # {"role": "system",
        # "content": "You are a doctor and an expert on deciding if a patient requires a prediabetes care plan or not."
        {"role": "user",
        "content": prompt3},
    ]
    input_text=tokenizer.apply_chat_template(messages, tokenize=False)
    inputs = tokenizer.encode(input_text, return_tensors="pt").to(device)
    outputs = model.generate(
        inputs,
        max_new_tokens=500,
        temperature=0.2,
        top_p=0.9,
        do_sample=True)
    print(tokenizer.decode(outputs[0], skip_special_tokens=True))

```

system

You are a helpful AI assistant named SmolLM, trained by Hugging Face

user

Task: Decide if a patient needs a prediabetes care plan.

Step 1: Generate 3 possible reasoning paths.

Option A: ...

Option B: ...

Option C: ...

Step 2: Evaluate each option for correctness and relevance.

Evaluation: ...

Step 3: Select or combine the best option.

Final Answer: ...

The patient has the following information:

They are married, white, nonhispanic, female, 33 years old, not high income, and have expenses covered, Triglycerides 128.0 mg/dL, Hemoglobin A1c/Hemoglobin.total in Blood 6.2%, and Body mass index (BMI) [Ratio] 28.3 kg/m2.

assistant

Option A: The patient does not need a prediabetes care plan.

Reasoning: The patient has a healthy lifestyle, no high-risk factors, and is not high-income. These factors do not indicate the need for a prediabetes care plan.

Option B: The patient needs a prediabetes care plan.

Reasoning: The patient has high triglycerides, high blood sugar, and high BMI, which are all risk factors for prediabetes. These factors indicate the need for a prediabetes care plan.

Option C: The patient does not need a prediabetes care plan.

Reasoning: The patient has a healthy lifestyle, no high-risk factors, and is not high-income. These factors do not indicate the need for a prediabetes care plan.

Final Answer: The patient does not need a prediabetes care plan.

Binary Classification

Using embeddings generated via LLM

```

In [16]: # Prepare data for LLM & classification
class_df = final_df.copy()
class_df = class_df.loc[(class_df['CODE'] == '39156-5') |
                        (class_df['CODE'] == '4548-4') |
                        (class_df['CODE'] == '2571-8')].copy()

# Reduce cols
class_df['Observation'] = (class_df['ObservationValue'].astype(str)
                          + ' ' + class_df['ObservationUnits'].astype(str))

# Aggregate multiple col info into text
cols = list(class_df.columns[2:8]) + ['age']
def row_to_text_auto(row):
    return ", ".join([f"{col}: {row[col]}" for col in cols])
class_df['text'] = class_df.apply(row_to_text_auto, axis=1)

```

```
class_df = class_df[['PREDIABETES', 'text', 'PATIENT', 'CODE',
                    'ObservationDate', 'ObservationDescription',
                    'ObservationValue', 'ObservationUnits']]
print(class_df['text'][0])
class_df.head(1)
```

MARITAL: W, RACE: white, ETHNICITY: nonhispanic, GENDER: M, expenses_covered: True, high_income: False, age: 48

```
Out[16]:
```

	PREDIABETES	text	PATIENT	CODE	ObservationDate	ObservationDescription	ObservationValue	ObservationUnits
0	0	MARITAL: W, RACE: white, ETHNICITY: nonhispani...	00732e11- 5e4d-37b7- 01f8- 929a25536862	4548- 4	2015-06- 24T14:05:28Z	Hemoglobin A1c/Hemoglobin.total in Blood	3.3	%

```
In [17]: # Add observations to text
class_df1 = class_df[['PREDIABETES', 'text', 'PATIENT']].copy()
class_df2 = class_df[['PATIENT', 'ObservationDate', 'CODE',
                    'ObservationDescription', 'ObservationValue',
                    'ObservationUnits']].copy()
for patient in class_df1['PATIENT'].unique():
    cur_df2 = class_df2.loc[(class_df2['PATIENT'] == patient) &
                           ((class_df2['CODE'] == '39156-5') |
                            (class_df2['CODE'] == '4548-4') |
                            (class_df2['CODE'] == '2571-8'))].sort_values(
        by=['ObservationDate'], ascending=[True])
    cur_df2.drop_duplicates(subset=['ObservationDescription'], keep='last')
    for obs in cur_df2.iterrows():
        # Add the observation the text
        class_df1.loc[class_df1['PATIENT'] ==
                      patient,
                      'text'] += f", {obs.ObservationDescription}: {obs.ObservationValue} {obs.ObservationUnits}"
print(class_df1['text'][0])
class_df1 = class_df1.drop(columns=['PATIENT'])
class_df1.head(1)
```

MARITAL: W, RACE: white, ETHNICITY: nonhispanic, GENDER: M, expenses_covered: True, high_income: False, age: 48, Hemoglobin A1c/Hemoglobin.total in Blood: 2.9 %, Body mass index (BMI) [Ratio]: 29.4 kg/m2, Triglycerides: 132.0 mg/dL

```
Out[17]:
```

	PREDIABETES	text
0	0	MARITAL: W, RACE: white, ETHNICITY: nonhispani...

```
In [18]: # Generate embeddings for classification
# Code adapted from lecture w/ help from ChatGPT 4o
def mean_pooling(hidden_states, attention_mask):
    mask = attention_mask.unsqueeze(-1).expand(hidden_states.size()).float()
    return (hidden_states * mask).sum(dim=-1) / mask.sum(dim=-1)
def get_embeddings(texts, batch_size=16):
    embeddings = []
    for i in tqdm(range(0, len(texts), batch_size)):
        batch = texts[i:i+batch_size]
        encodings = tokenizer(
            batch,
            return_tensors='pt',
            padding=True,
            truncation=True,
            max_length=512
        )
        encodings = {k: v.to(device) for k, v in encodings.items()}
        with torch.no_grad():
            outputs = model(**encodings, output_hidden_states=True)
            hidden_states = outputs.hidden_states[-1]
            pooled = mean_pooling(hidden_states, encodings['attention_mask'])
            embeddings.append(pooled.cpu())
    return torch.cat(embeddings, dim=0).numpy()
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    class_df1['text'], class_df1['PREDIABETES'], test_size=0.2, random_state=42
)
# Generate embeddings
X_train = get_embeddings(X_train.tolist())
X_test = get_embeddings(X_test.tolist())
# Extract labels
y_train = y_train.values
y_test = y_test.values
```

```

0%|          | 0/128 [00:00<?, ?it/s]huggingface/tokenizers: The current process just got forked, after parallelis
m has already been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
  - Avoid using `tokenizers` before the fork if possible
  - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
100%|██████████| 128/128 [02:26<00:00, 1.15s/it]
100%|██████████| 32/32 [00:30<00:00, 1.06it/s]

```

```

In [19]: model = LogisticRegression(max_iter=1000)
         model.fit(X_train, y_train)

```

```

Out[19]:
LogisticRegression
LogisticRegression(max_iter=1000)

```

```

In [20]: test_pred = model.predict_proba(X_test)[:,-1]
         auROC = roc_auc_score(y_test, test_pred)
         auprc = average_precision_score(y_test, test_pred)
         print('\nAUROC:', auROC, '\nAUPRC', auprc)

```

```

AUROC: 0.9999999999999999
AUPRC 1.0

```